

Колекції

Частина 4. Множини

Множина (змінюваний тип **set**) — це невпорядкований набір (колекція) унікальних (такі, що не повторюються) елементів незмінюваних типів. Крім операцій, характерних для змінюваних структур даних, таких як списки і словники, множини підтримують притаманні їхнім математичним відповідникам операції перетину, об'єднання та різниці.

Подання множин

Літералом множини в Python є послідовність об'єктів, записаних через кому у фігурних дужках:

```
digits = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
rgbColors = {'red', 'green', 'blue'}
assorty = {1, 'one', (1, 1.)}
emptySet = set()
```

Оскільки елементами множини можуть бути тільки незмінювані типи, то спроба створити множину, яка містить список, словник або іншу множину³, викличе помилку, наприклад:

```
wrongSet = {1, [2, 3]}          # TypeError: unhashable type: 'list'
```

Операції над множинами

Використовуючи оператори "-", "|", "&", "^", ">" та ін. над множинами можна виконувати операції, які не підтримуються іншими колекціями.

```
rgbColors = {'red', 'green', 'blue'}
trafficColors = {'red', 'yellow', 'green'}

rgbColors - trafficColors
# {'blue'} (різниця множин)

rgbColors | trafficColors
# {'red', 'green', 'blue', 'yellow'} # (об'єднання множин)
```

```
rgbColors & trafficColors      # {'red', 'green'} (перетин множин)

rgbColors ^ trafficColors     # {'yellow', 'blue'}
# (симетрична різниця: елементи, які є тільки в одній з множин)

rgbColors > trafficColors
# False (перевірка, чи перша множина є надмножиною другої)

rgbColors < trafficColors
# False (перевірка, чи перша множина є підмножиною другої)

'blue' in rgbColors # True (перевірка на входження оператором in)
```

Функції для роботи з множинами та методи множин

Функція `len()` повертає кількість елементів множини:

```
len(assorty)      # 3 (множина assorty містить число, рядок і кортеж)
```

Функцію `set()`, яка створює множину з послідовності (колекції), часто використовують для видалення дублікатів:

```
from random import randint

bits = [randint(0,1) for i in range(100)]
print(bits)                # [1, 0, 1, 1, 0, 0, 0, 0, 0, 1, ...]
bitSet = set(bits)
print(bitSet)               # {0, 1}

print(list(set('Антарктида'))
# ['p', 'a', 'A', 'н', 'и', 'к', 'т', 'д']
```

Функція `sorted()` повертає відсортований список елементів множини:

```
print(sorted(rgbColors)) # ['blue', 'red', 'yellow']
```

Методи множин:

<code>A.add(x)</code>	# додавання елемента x
<code>A.remove(x)</code>	# вилучення елемента x з множини
	# (викликає помилку за відсутності елемента)
<code>A.discard(x)</code>	# вилучення елемента x, якщо такий є
<code>A.pop()</code>	# вилучення довільного елемента

```
A.clear()           # очищення множини
A.union(B)          # об'єднання множин  $A \cup B$ ; аналог операції  $A \mid B$ 
A.intersection(B)   # перетин множин  $A \cap B$ ; аналог  $A \& B$ 
A.difference(B)     # різниця множин  $A \setminus B$ ; аналог  $A - B$ 
A.issubset(B)       # чи A є підмножиною B; аналог  $A < B$ 
```

Генератори множин

Як і розглянуті вище колекції, множини підтримують генерування шляхом конструкції (інструкція `if` може бути опущена)

```
{expression for item in collection if condition}
```

Приклади генерування множини квадратів парних двоцифрових натуральних чисел:

```
s = {n**2 for n in range(10, 100) if n%2 == 0}
s = {n**2 for n in range(10, 100, 2)}
```

Проте, на відміну від списків, порядок розташування елементів у множині буде порушений.
